Ztest, Twister und deterministische Fehler-Injektion

Testdesign, Live-Testlauf und gemeinsame Auswertung

4.07

Testebenen und bewusste Grenzen

LOGIKTEST

Reine Fachfunktion

ohne Sensorgerät

schnell und isoliert

Grenzen und Regeln

INTEGRATIONSTEST

Zephyr + native_sim

Treiberpfad und Fixture

Queue und Threads

reproduzierbare Fehler

SYSTEMABNAHME

Gesamtes Produkt

reale Umgebung später

Hardwareeigenschaften

Ende-zu-Ende-Verhalten

Ein grüner Test belegt nur die Ebene, die seine Fixture tatsächlich ausführt.

Ztest-Fixture als reproduzierbarer Rahmen

TEST-LEBENSZYKLUS

```
suite → setup → before → test → after → teardown
```

Suite	gemeinsame Fixture und Szenariogruppe definieren
Setup	einmalige Ressourcen für die Suite vorbereiten
Before / After	Zustand vor jedem Test setzen und danach kontrolliert bereinigen
Test	eine Aktion ausführen und beobachtbaren Zustand prüfen
Teardown	Suite-Ressourcen freigeben und Endzustand sichern

Jeder Test beginnt aus einem bekannten Zustand und endet ohne Restwirkung.

Twister-Szenario für native_sim

TESTCASE.YAML

```
tests:
  day4.motion_pipeline:
    platform_allow: native_sim
    tags: sensor pipeline
    timeout: 30
```

01

Plattform

native_sim führt Treiber, Queue, Threads und Fixture reproduzierbar aus.

02

Aussage

Szenario, Exit-Status und geprüfte Endzustände werden dokumentiert.

Twister macht den Lauf wiederholbar – die Assertions bestimmen seine Aussagekraft.

Testdaten getrennt vom Anwendungscode

NAMEN UND FIXTURES

```
fixture.valid_edge
fixture.sensor_error
fixture.sequence_gap
fixture.queue_full

test_<scenario>_<action>_<effect>
```

01 Szenario

Testdaten und Emulatorzustand eindeutig beschreiben.

02 Aktion

genau einen Auslöser oder Verarbeitungsschritt ausführen.

03 Wirkung

Endzustand, Rückgabewert und Zählerdifferenz prüfen.

Testnamen erzählen Ursache und erwartete Wirkung – nicht die Implementierung.

Deterministische Fehler-Injektion

01	`Sensorpfad`	Sensorfehler oder nicht unterstützten Channel gezielt zurückgeben
02	`Probe-Zustand`	Sequenzlücke und veraltete Probe mit festem Zeitbezug erzeugen
03	`Queue`	begrenzte Tiefe füllen und gewählte Überlaststrategie auslösen

Der Fehler wird vorbereitet, ausgelöst und über Zustand sowie Zähler nachgewiesen.

Was die Tests tatsächlich prüfen

ASSERTIONS

`Zustand vorher → Aktion → Zustand nachher → Zählerdifferenz`

GÜLTIG + GRENZWERT	Qualität und normalisierte Messwerte stimmen
FEHLER + CHANNEL	SENSOR_ERROR und -ENOTSUP bleiben unterscheidbar
LÜCKE + STALE	Sequenzlücke und Datenalter werden erkannt
QUEUE-ÜBERLAST	Drop-Strategie und Zähleränderung entsprechen dem Vertrag

Endzustände und Zähler sind stabiler als die Reihenfolge einzelner Logzeilen.

Abnahme: Logiktest und Integrationstest

TESTLAUF

```
$ west twister -T tests/logic -p native_sim
$ west twister -T tests/integration -p native_sim

PASS  validation_valid_edge
PASS  pipeline_sensor_error
PASS  pipeline_queue_full
```

01 Nachweisen

mindestens ein isolierter Logiktest

02 Einordnen

ein native_sim-Test mit Fixture und klarer Aussagegrenze

Ergebnis: Fehlerfälle sind reproduzierbar und ihre Wirkung ist automatisiert geprüft.